

Backend Document: Vibe Problem Solver

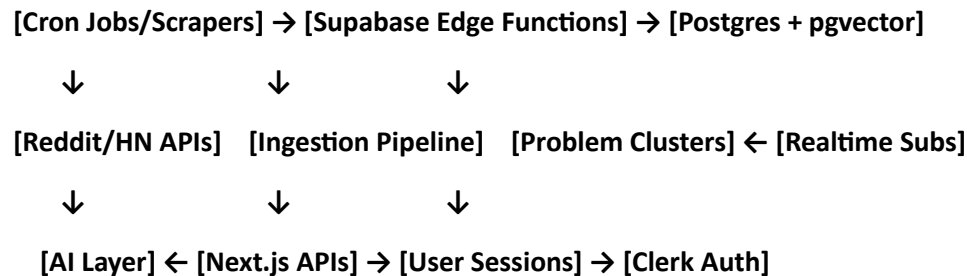
Overview

The backend architecture uses Supabase as the central platform (Postgres database + auth + edge functions) with Next.js server actions handling API orchestration. Core components include problem ingestion pipeline, Pain Index computation, AI service layer (Claude/GPT), and realtime subscriptions. Design supports 1K DAU with auto-scaling, rate limiting, and RLS security.

Deployment: Supabase hosted project + Vercel edge functions. No custom servers needed for MVP.

Backend Architecture Diagram

text



Core Components

1. Data Ingestion Pipeline (Edge Functions)

Purpose: Scrape forums → extract problems → compute Pain Index → store clusters.

Supabase Edge Function (supabase/functions/ingest/index.ts):

typescript

```
Deno.serve(async (req) => {  
  const sources = await Promise.all([  
    scrapeReddit('r/reactjs'),
```

```
scrapeHackerNews(),  
fetchPerplexityProblems()  
])
```

```
// Extract + cluster
```

```
const clusters = await processMentions(sources)
```

```
// Compute Pain Index
```

```
for (const cluster of clusters) {  
  cluster.painIndex = calculatePi(cluster)  
  await supabase.from('problem_clusters').upsert(cluster)  
}
```

```
return new Response('Ingestion complete')
```

```
}}
```

Cron Schedule: Hourly via Supabase cron (0 * * * *).

2. Pain Index Computation

Formula Implementation:

```
sql
```

```
-- Postgres function for Pi calculation
```

```

CREATE OR REPLACE FUNCTION calculate_pain_index(
  volume INTEGER,
  sentiment FLOAT,
  age_hours INTEGER,
  authority FLOAT
) RETURNS FLOAT AS $$
  SELECT (volume * sentiment / GREATEST(age_hours, 1)) * authority;
$$ LANGUAGE SQL IMMUTABLE;

```

Storage Triggers (Auto-compute on insert):

sql

```

CREATE TRIGGER pain_index_trigger
  AFTER INSERT OR UPDATE ON raw_mentions
  FOR EACH ROW EXECUTE FUNCTION update_cluster_pi();

```

3. API Layer (Next.js Server Actions)

Primary Endpoints (/app/api/):

Endpoint	Method	Purpose	Rate Limit
/api/problems	GET	Fetch ranked feed	20/min

Endpoint	Method	Purpose	Rate Limit
/api/[id]/solution	POST	Generate layers	10/min
/api/roadmap	POST	Create phased plan	5/min
/api/docs/[type]	POST	Generate PRD/TechDoc	5/min

Example Server Action:

typescript

'use server'

import { createClient } from '@/utils/supabase/server'

import { callClaude } from '@/lib/ai'

export async function generateSolution(problemId: string) {

const supabase = createClient()

// Fetch problem

const { data: problem } = await supabase

.from('problem_clusters')

.select('*')

```
.eq('id', problemId)
```

```
.single()
```

```
// Generate layers
```

```
const solution = await callClaude({
```

```
  prompt: `Generate 6-layer solution for: ${problem.canonical_text}`,
```

```
  model: 'claude-3-5-sonnet'
```

```
})
```

```
// Store + realtime broadcast
```

```
const { data } = await supabase
```

```
.from('solutions')
```

```
.insert({ problem_id: problemId, layers: solution })
```

```
.select()
```

```
return data
```

```
}
```

4. Database Schema (Prisma + Supabase)

Key Tables (from Tech Doc):

text

```

model ProblemCluster {
  id      String @id @default(cuid())
  canonicalText String
  painIndex Float
  embedding Unsupported("vector(1536)") // pgvector
  sources  Json[]
}

```

```

model Solution {
  id      String @id @default(cuid())
  problemId String
  layers   Json // 6-layer structure
  techStack Json
  problemCluster ProblemCluster @relation(fields: [problemId], references: [id])
}

```

RLS Policies:

```
sql
```

```
-- Users see problems matching their domains
```

```
CREATE POLICY "Domain-restricted problems" ON problem_clusters
```

```
FOR SELECT USING (
```

```
auth.uid() IN (  
  SELECT user_id FROM user_domains  
  WHERE domains @> array[domain_tags]  
)  
);
```

5. AI Service Layer

Model Orchestration (lib/ai.ts):

typescript

```
export async function callClaude({ prompt, model = 'claude-3-5-sonnet' }) {  
  // Cache check first  
  const cache = await kv.get(['ai', hash(prompt)])  
  if (cache) return cache  
  
  const response = await fetch('https://api.anthropic.com/v1/messages', {  
    headers: { 'x-api-key': env.CLAUDE_API_KEY },  
    body: JSON.stringify({ model, messages: [{ role: 'user', content: prompt }] })  
  })  
  
  const result = await response.json()  
  await kv.set(['ai', hash(prompt)], result, { ex: 3600 }) // 1hr cache
```

```
return result.content[0].text
```

```
}
```

Fallback Chain: Claude → GPT-5o → Template-only.

6. Background Workers

Supabase Edge Functions:

- ingest-problems: Hourly scraping + clustering
- recompute-pi: Daily Pain Index refresh
- user-reports: Weekly personalized problem digests

Queue Implementation: Supabase public.jobs table + cron.

7. Rate Limiting & Security

Implementation:

typescript

// Upstash Redis (Vercel integration)

```
const rateLimit = new RateLimit({  
  redis: redis,  
  limiter: RateLimiter.memoryLimiter({  
    points: 10, duration: 60 // 10/min  
  })  
})
```



```
export async function GET(request: Request) {  
  if (!await rateLimit.check(request)) {  
    return Response.json({ error: 'Rate limited' }, { status: 429 })  
  }  
  // ...  
}
```

Security Checklist:

- API keys in Vercel env vars only
- Zod validation on all inputs
- RLS enabled on all tables
- Clerk middleware for auth
- Sentry error capture
- Rate limits: ingestion (50/hr), generation (10/min)

8. Monitoring & Observability

text

Sentry (errors) ← All server actions

Vercel Analytics ← API response times

Supabase Logs ← DB queries

PostHog ← User events (roadmap_created, doc_exported)

Alerts:

- LLM cost > \$50/day
- Pi computation > 5min
- Error rate > 2%

9. Deployment & Environment

text

Vercel Environment Variables:

NEXT_PUBLIC_SUPABASE_URL=...

SUPABASE_SERVICE_KEY=... # Edge functions

CLAUDE_API_KEY=...

STRIPE_SECRET_KEY=... # Post-MVP

Supabase Settings:

- pgvector extension enabled
- Compute: Add-ons for >500 DAU
- Point-in-time recovery: 7 days

10. API Contract (OpenAPI Spec)

text

paths:

/api/problems:

get:

parameters:

- name: domain

schema: { type: string, enum: [tech, finance, education] }

- name: limit

schema: { type: integer, default: 20 }

responses:

'200':

content:

application/json:

schema:

type: array

items:

type: object

properties:

id: { type: string }

canonicalText: { type: string }

painIndex: { type: number, minimum: 0, maximum: 10 }

Scalability Plan

DAU	Infra	Cost
100	Supabase Free + Vercel Hobby	\$0
1K	Supabase Pro + Compute Add-on	\$50/mo
10K	Read replicas + CDN	\$500/mo

Risks & Mitigations

Risk	Mitigation
Scraper blocked	Multiple sources + polite backoffs
LLM hallucination	Template-first + confidence scoring
DB hotspot	Composite indexes on domain_tags + pain_index

Next Steps: Deploy Supabase project (Day 1), implement ingestion cron (Day 3), load test APIs (Day 5). Export as MD/PDF via prisma doc generate.